Platform Migrations Without the Downtime

An Engineering Playbook

Sai Krishna Reddy Ala
Senior Applications Developer, Bath & Body Works
Conf42 Platform Engineering 2026

Agenda

1	**Why zero-downtime migration is hard**	The problem and why big-bang cutovers fail
2	**Three core patterns**	Dual-write, bidirectional sync, change data capture
3	**Choosing a pattern**	Matching the pattern to the migration
4	**Readiness framework**	Five dimensions to assess before cutover
5	**Platform capability**	Building it once, reusing it everywhere

Platform teams now own the hardest project

Replacing a legacy system the business depends on every hour of every day — with zero tolerance for downtime.

1

Enterprise order management

Orders can't stop flowing during cutover

2

Healthcare claims

Accuracy and auditability are non-negotiable

3

CRM platforms

Customer data must stay consistent across systems

Why big-bang cutovers fail

One weekend. One chance.

A single switch-over puts every risk on the same moment, with no live comparison and no easy way back.

Hidden data drift

Differences surface only after go-live

No real-traffic validation

Test data never matches production behavior

Painful rollback

Data written after cutover is hard to unwind

Business-wide blast radius

Every user and integration is hit at once

Three patterns to build as reusable capability

1

Dual-write parallel run

Run legacy and new side by side; compare before you trust

2

Bidirectional synchronization

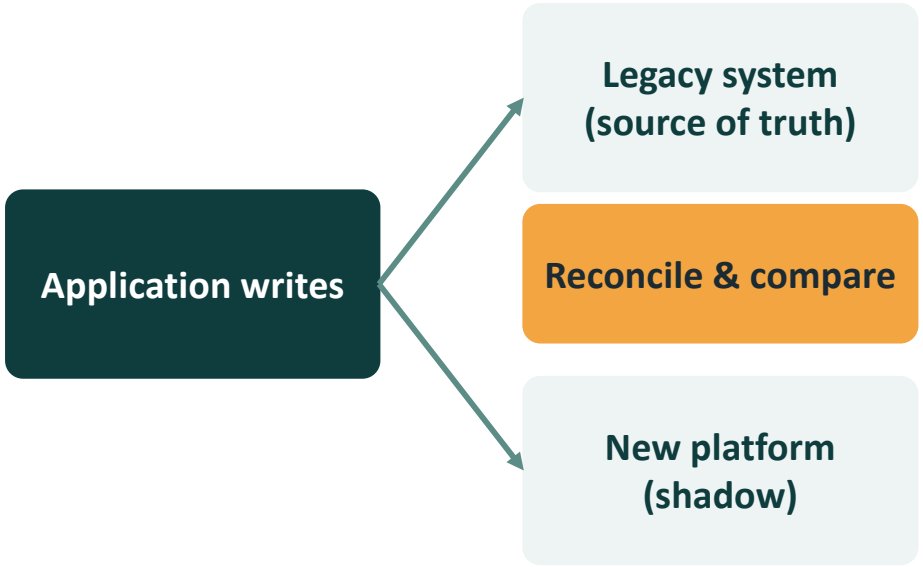
Keep legacy and new consistent while both are live

3

Change data capture (CDC)

Move data incrementally, at scale, from the source log

Pattern 1: Dual-write parallel run



- Write to both systems; read from legacy until parity is proven
- Automated reconciliation flags divergence early
- Flip reads gradually — and keep a fast rollback path

Running the parallel run in phases



Shadow

New platform receives writes; legacy serves all reads

Compare

Reconcile outputs and fix every divergence

Canary reads

Route a small share of reads to the new platform

Flip

New platform becomes the source of truth

Retire

Legacy kept as fallback, then decommissioned

Pattern 2: Bidirectional synchronization



- Conflict rules
- Loop prevention
- Idempotent writes

Both systems stay live and consistent while users and services move over in waves.

Bidirectional sync: design rules

1

Resolve conflicts deterministically

Define which system wins per field, or use timestamps and versions

2

Prevent echo loops

Tag the origin of each change so it is never synced back

3

Make every write idempotent

Retries and replays must never create duplicates

4

Preserve ordering

Apply changes per entity in the order they happened

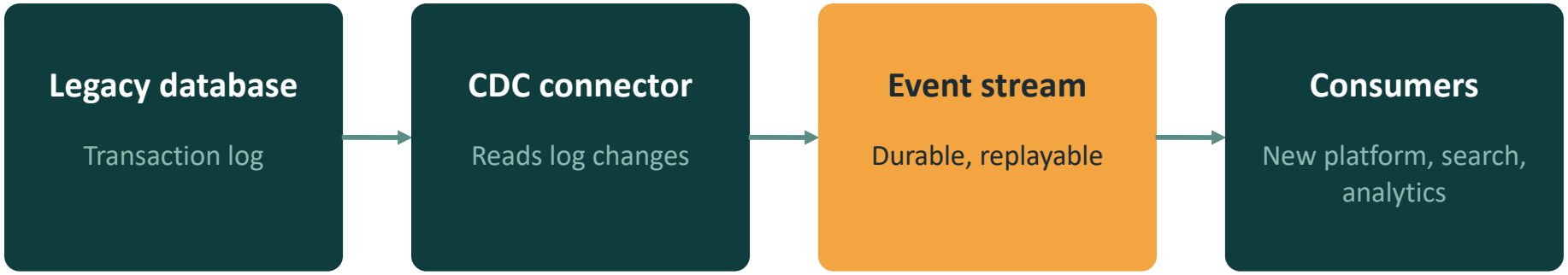
Pattern 3: Change data capture

Hours → ms

Data latency reduction possible with event-driven approaches (Forrester-cited)

- Stream changes from the source database log — no batch windows
- Incremental, scalable data movement without loading the legacy app
- Replayable events give a safe backfill and recovery path

Anatomy of a CDC pipeline



- Initial snapshot, then continuous incremental changes
- Schema changes handled as versioned events
- Monitor lag end to end — it is your migration's heartbeat

Choosing the right pattern

	Dual-write parallel run	Bidirectional sync	Change data capture
Best for	Proving the new system on real traffic	Long coexistence with users on both	High-volume incremental data movement
Data flow	App → both systems	Legacy ⇄ new	Legacy log → stream → targets
Main risk	Write divergence	Conflicts and loops	Lag and schema drift
Legacy changes needed	App write path	Sync hooks or adapters	Minimal — reads the log

Is your platform ready for the cutover?

A five-dimension maturity framework

1

Latency

How fresh is data across systems?

2

Reliability

Does sync survive failure and replay?

3

Bidirectionality

Can changes flow both ways safely?

4

Scalability

Does it hold at peak volume?

5

Migration safety

Can you verify, roll back, and cut over in steps?

Build it once, as platform capability

Project by project

- Every team reinvents sync and reconciliation
- Inconsistent safety and rollback
- Each cutover is a high-risk event

As platform capability

- Dual-write, sync, and CDC as paved roads
- Readiness scored on five dimensions
- Cutovers become routine, not heroic

Thank you

Sai Krishna Reddy Ala

Senior Applications Developer, Bath & Body Works

Questions welcome